

Computer Organization & Design

Lab 8

Tsiantos Dimitrios & Romanidis Angelos
(3796 & 3980)

December 2024

1 Simulation of Cache Memory Operation

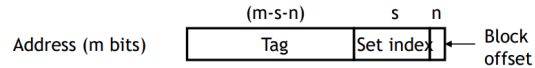
- Main Memory: 512 Bytes
- 2-way Associative Cache: 64 Bytes
- LRU Replacement Strategy
- 4-byte Cache Blocks

Solution:

The cache memory is 2-way associative, so each set fits 2 blocks. The size of each block is 4 bytes, meaning the total number of blocks is 16, resulting in 8 sets.

Main Memory Size: 2^9 , which means 9 address bits.

Address breakdown in a set associative cache:



- $m = 9$ bits
- Block Offset = 2 bits, since $2^2 = 4$ (block size = 4).
- Set Index = 3 bits, since $2^3 = 8$ (number of sets).

Therefore, we conclude that the tag = 4 bits.

The block offset bits show us which byte of the block we want to access, and each block contains 4 bytes.

DecAddr	207	212	416	289	204	222	461	109	280	303	2	480
BinAddr	011001111	011010100	110100000	100100001	011001100	011011110	111001101	001101101	100011000	100101111	000000010	111100000

Initially, the cache memory is empty, so the valid bits of all blocks are initialized to 0.

So, after the accesses, the cache memory contains:

Set	Block 1: Addr / Offset	Block 2: Addr / Offset	Valid Bit (B1/B2)	Tag (B1/B2)	Data (B1/B2)
000	2 / 10	480 / 00	1 / 1	0000 / 1111	Mem[2] / Mem[480]
001	- / -	- / -	0 / 0	- / -	- / -
010	- / -	- / -	0 / 0	- / -	- / -
011	303 / 11	109 / 01	1 / 1	1001 / 0011	Mem[303] / Mem[109]
100	- / -	- / -	0 / 0	- / -	- / -
101	212 / 00	- / -	1 / 0	0110 / -	Mem[212] / -
110	280 / 00	- / -	1 / 0	1000 / -	Mem[280] / -
111	222 / 10	- / -	1 / 0	0110 / -	Mem[222] / -

2 Code Behavior in Memory Hierarchy

$a[0][0]$	$a[0][1]$	$a[0][2]$	...	$a[0][99]$
$a[1][0]$	$a[1][1]$	$a[1][2]$	...	$a[1][99]$

Array a is written row by row, so we take advantage of its spatial locality. Since the L1 DCache block size is 32 bytes, we can fit 4 elements of array a per cache block, as the arrays are doubles and each element is 8 bytes. Array a is stored row by row, so the first 4 elements (i.e., elements $a[0][0]$, $a[0][1]$, $a[0][2]$, $a[0][3]$) will be loaded into the first cache block. When we access $a[0][4]$, $a[0][5]$, $a[0][6]$, $a[0][7]$, they will be cache hits because the entire block was fetched during the access of the previous memory location. Therefore, data from the same row is loaded into cache blocks, and subsequent accesses in the same row lead to hits, as the data is already in the cache. Since we have a total of 3×100 accesses to array a in our code, there will be $300/4 = 75$ misses due to array a .

$b[0][0]$	$b[0][1]$	$b[0][2]$
$b[1][0]$	$b[1][1]$	$b[1][2]$
...	...	...
$b[100][0]$	$b[100][1]$	$b[100][2]$

Array b is accessed by columns, and the same elements are accessed regardless of the outer loop index. The first execution of the inner for loop creates 4 misses ($b[0][0]$, $b[1][0]$, $b[2][0]$, $b[3][0]$). The second execution of the inner for loop creates only 1 miss at $b[4][0]$ because $b[1][0]$, $b[2][0]$, $b[3][0]$ have already been transferred from the first iteration. The same happens for all array indices from 1...97 (when the inner for loop index = 96, we check if we will fetch $b[100][3]$). Consequently, the access of array b creates $4 + 97 = 101$ read misses in a total of $3 \times 3 \times 100 = 900$ accesses. Overall, the execution of the above code creates $75 + 101 = 176$ misses. The total miss rate is $176/(900 + 300) = 14.6\%$.

3 Performance Analysis of the K-Means Program for Image Compression

a) By running the original kmeans algorithm with the four images as inputs for $k = \{2, 3, \dots, 8\}$, we observe some differences in the execution time of each program.

It is necessary to run the algorithm several times, as kmeans has a degree of randomness due to the way it selects the means.

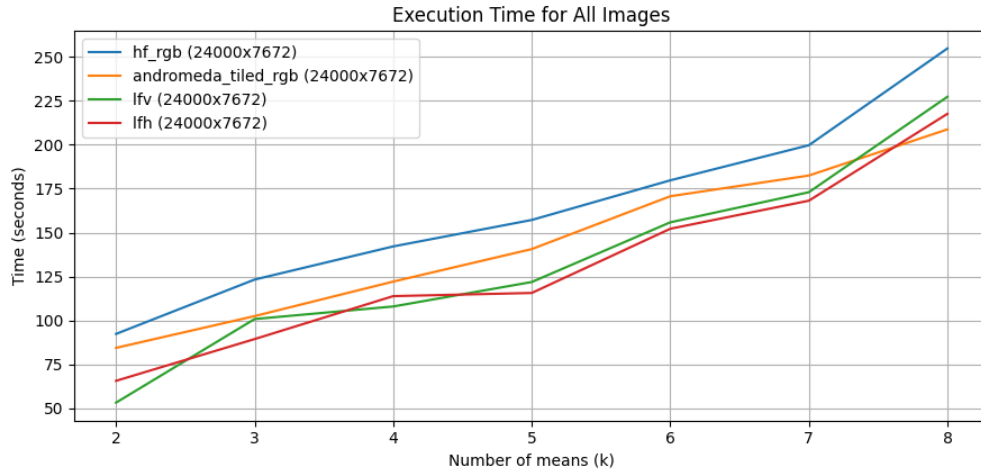


Figure 1: K-means execution time on a 12th Gen Intel® Core™ i5-1235U × 12 processor on a single core.

Almost all tests had similar results with negligible differences (Figures 13 - 16). The image `hf_rgb.bmp` executes slower, the image `andromeda_tiled_rgb.bmp` faster, and `lfh.bmp` and `lfv.bmp` had on average the same execution time, lower than the previous two. From these measurements we can conclude that:

$$time(lfh.bmp) = time(lfv.bmp) < time(andromeda_tiled_rgb.bmp) < time(hf_rgb.bmp)$$

In figures 14, 15, and 16 we see abnormal execution times at certain k for `lfv.bmp` due to a Floating Point Exception.

Although the images have exactly the same dimensions, the execution time differs. This is due to the variety of colors in each image. Images with more distinct colors have a longer execution time compared to those with fewer.

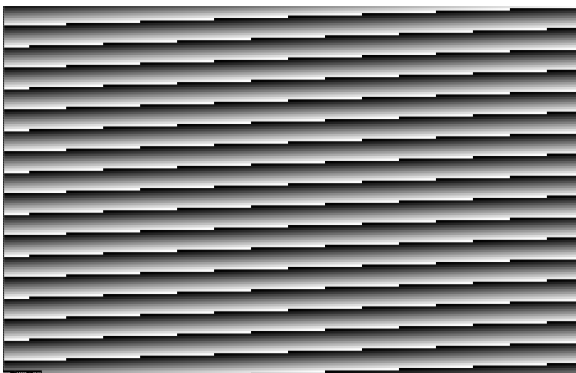


Figure 2: `lfh.bmp` (zoomed)

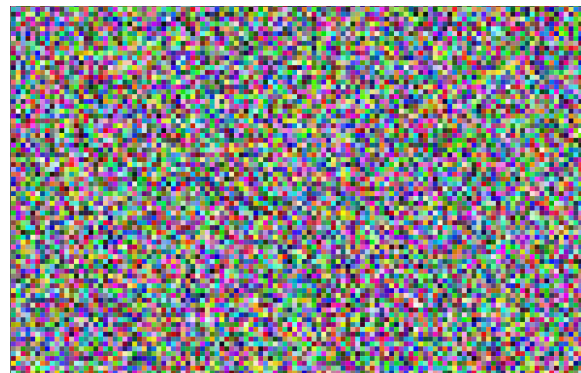


Figure 3: `hf_rgb.bmp` (zoomed)

Specifically, the algorithm needs more iterations to "stabilize" the means to an RGB value in the image with the most colors. This results in each pixel frequently changing which cluster it belongs to in each iteration. This creates a problem for the processor's branch predictor, which makes many mispredictions, as whether a branch will be taken is impossible to predict with any strategy when colors are random.

This observation can be confirmed by the VTune program, with the Microarchitecture Exploration check showing a significantly increased branch misprediction rate in the photo with random colors. Thus, the processor will flush several machine cycles, significantly delaying the code execution.

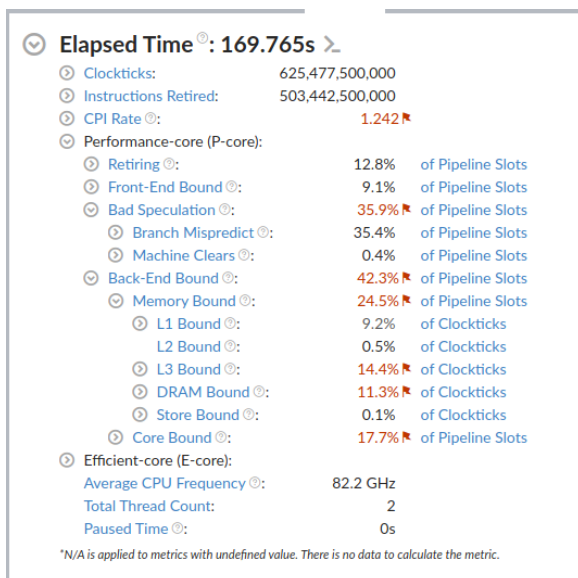


Figure 4: Measurements of the random color image

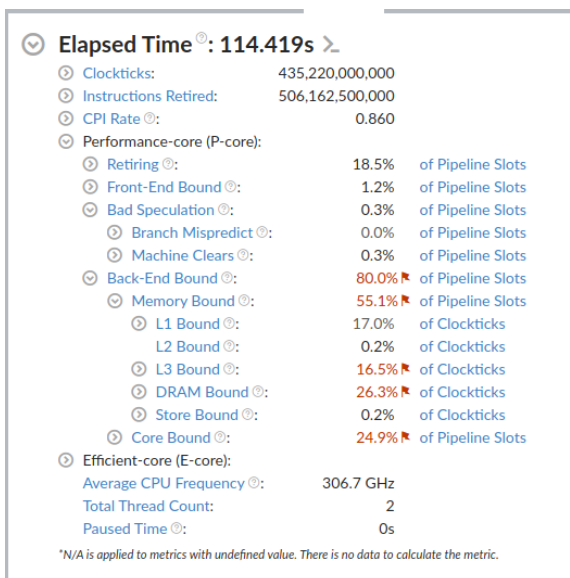


Figure 5: Measurements of the low color image

The very high Branch Misprediction rate (35.4%) of the random color image increases the CPI to 1.242 and makes execution significantly slower. It is worth noting that the Back-End Bound of this image is almost half that of the image with few colors. This suggests that while the processor is flushing, it utilizes this time by simultaneously fetching data from memory. Since the flush occurs due to misprediction and cache & memory misses are not responsible, the Back-End Bound metrics are low in the execution of the random color image.

b) When we provide the images `hf_rgb_p2.bmp` and `hf_rgb_p2p1.bmp` as input to the original kmeans program, although they are very similar in content, we observe that they do not run at the same speed.

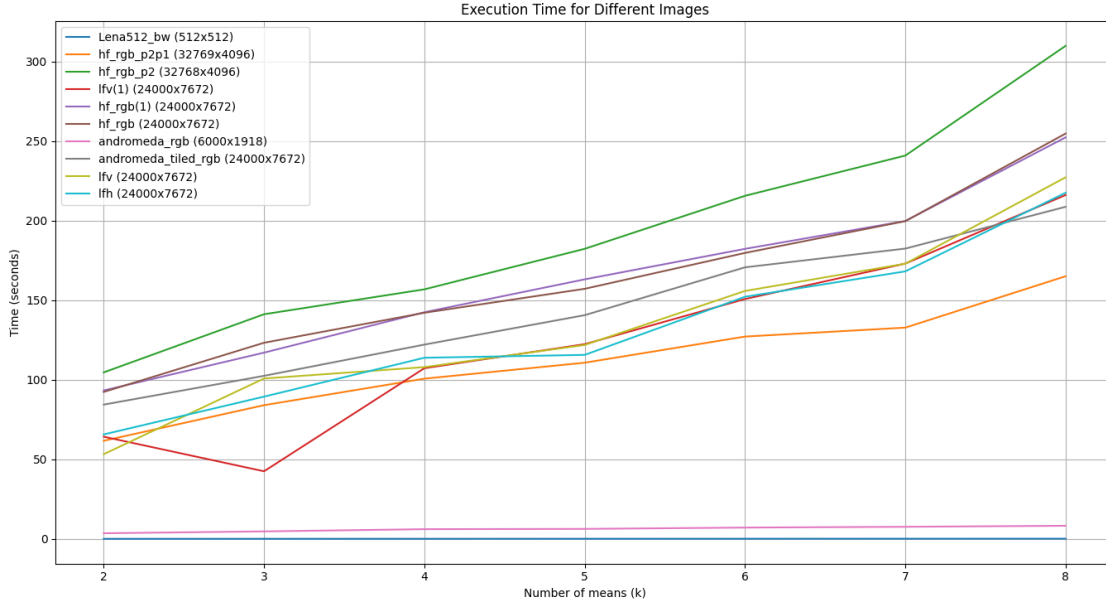


Figure 6: Execution time for all images. `hf_rgb_p2.bmp` (green) and `hf_rgb_p2p1.bmp` (orange)

The `p2` is the slowest of all images, while `p2p1` is the fastest for these specific dimensions. The only difference between the two is that `p2p1` has one extra column compared to `p2`, resulting in `p2` having a width that is a power of 2 ($p2p1 : 32769 \times 4096$ and $p2 : 32768 \times 4096$).

The reason this happens is the structure and operation of the cache memory. Because image data starts at a memory location that is a multiple of a power of 2, the image rows correspond to the same cache lines. This creates the phenomenon of cache aliasing, where the same memory locations compete for the same space in the cache, leading to frequent evictions and poor cache performance.

c) To improve program performance, changes can be made to the memory access method to allow for more efficient pre-fetching. Specifically, the algorithm needs access to all pixels of the image in step 2 of the cluster table and in steps 3 and 5. This process can be accelerated if the table is traversed first by rows and then by columns:

```
for (row = 0; row < height; ++row) {
    for (col = 0; col < width; ++col) {
        // ...row and col data accesses
    }
}
```

The code above is more efficient because when an element of the array is requested, the system performs prefetching for several neighboring memory locations. Because prefetching works on consecutive memory locations in sequence, traversing the array by rows ensures the use of memory locations that have already been loaded, increasing temporal locality. Conversely, traversing by columns is likely not to exploit the data already loaded, especially for large arrays (Figure 7).

Additionally, performance can be further improved by avoiding repeated access to struct fields. When the struct is large, each access causes a prefetch of the entire struct, which can be very slow. Instead, it is more efficient to store the values of the required fields in local variables.

If these values must remain in the struct for various reasons—such as better organization and code readability—as in our case, the struct fields can be organized into sub-structs. This way, the size of each accessed struct is limited, reducing the memory locations being prefetched, increasing spatial locality, and improving program performance.

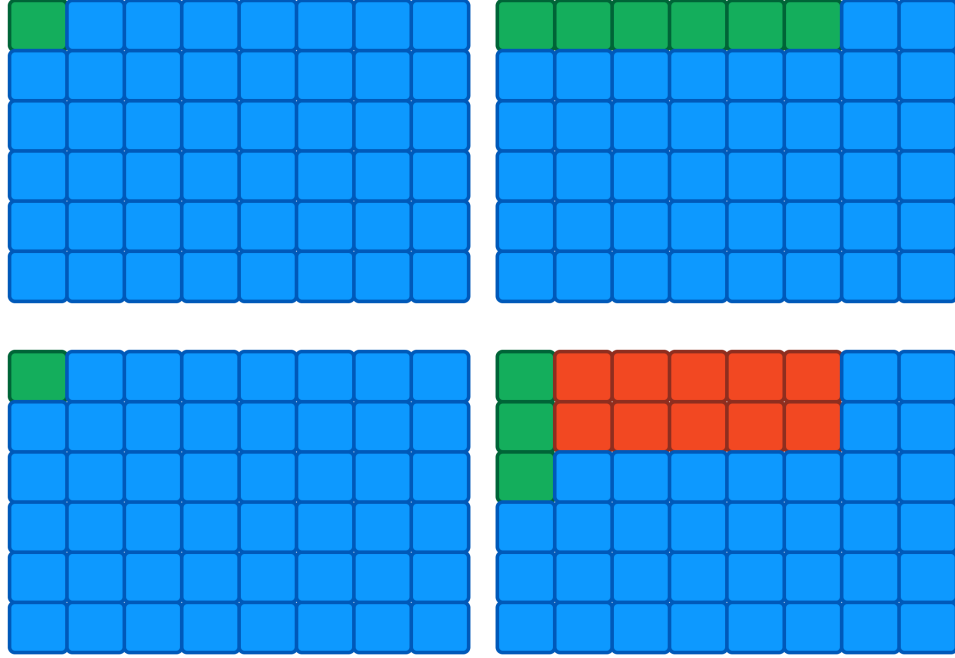


Figure 7: Example of accessing the first two elements of a two-dimensional array in row-major vs. column-major order. Red squares show data in the cache that is not read immediately.

Another optimization that can be made in the kmeans code is the calculation of the distance of elements from the means. Euclidean distance is quite slow as it uses square roots, multiplications, additions, and subtractions.

$$d_{\text{Euclidean}} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

In C code:

```
dist = (UINT) sqrt((r-means[i].r)*(r-means[i].r)
                  + (g-means[i].g)*(g-means[i].g)
                  + (b-means[i].b)*(b-means[i].b));
```

For the needs of kmeans, we could entirely remove the `sqrt` function, as we are not interested in the exact distance value, as long as we know if the new distance is smaller than the previous one:

$$\sqrt{\text{dist}} < \sqrt{\text{minDist}} \iff (\sqrt{\text{dist}})^2 < (\sqrt{\text{minDist}})^2 \iff \text{dist} < \text{minDist}$$

Even without square roots, the code remains slow due to multiplications (according to VTune Hotspot analysis).

For this reason, we can use the Manhattan distance, which uses only absolute values, three additions, and subtractions:

$$d_{\text{Manhattan}} = \sum_{i=1}^n |x_i - y_i|$$

In C code:

```
dist = abs(r - means[i].r) + abs(g - means[i].g) + abs(b - means[i].b);
```

It is worth noting that although Manhattan distance is faster than Euclidean, it has slightly worse results in image compression [5, 2]. In high-resolution images, however, the difference in quality is negligible.

Once we apply the above code optimization tactics, along with some other logical optimizations such as removing redundant operations, we observe that the execution time has been significantly reduced, by about 3.5 times.

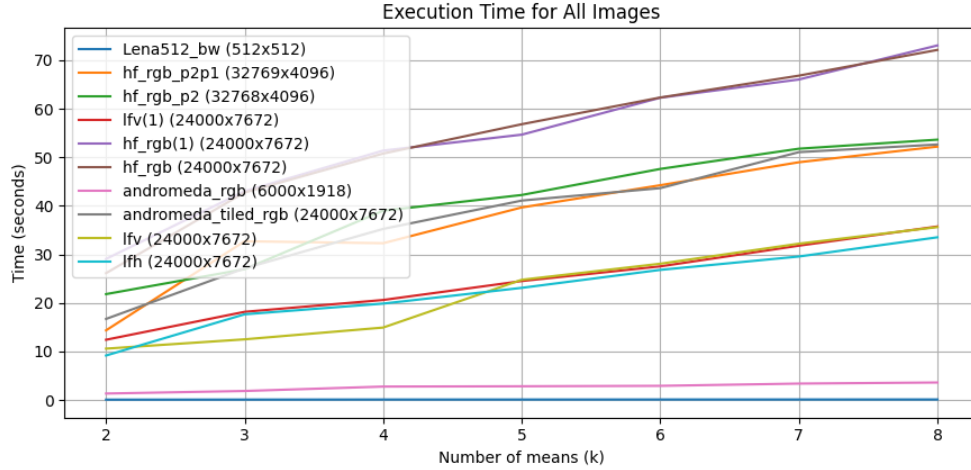


Figure 8: Execution time for each image with optimized code.

Besides the reduction in execution time for all images, we also notice that time is directly correlated with color variety and image dimensions. We see that images with random colors and higher resolution execute the slowest, followed by images of lower resolution but high color variety, then images with fewer colors, and finally images with the smallest dimensions.

In the next two pages, the misses in L1 & LLC caches for the original and optimized code are provided. The optimized code has a much lower number of misses, confirming the optimizations.

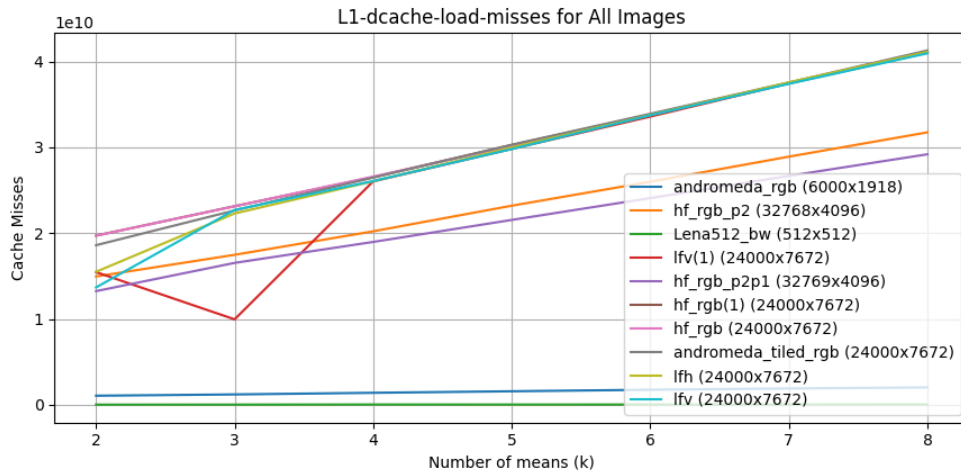


Figure 9: L1 cache misses of the original code

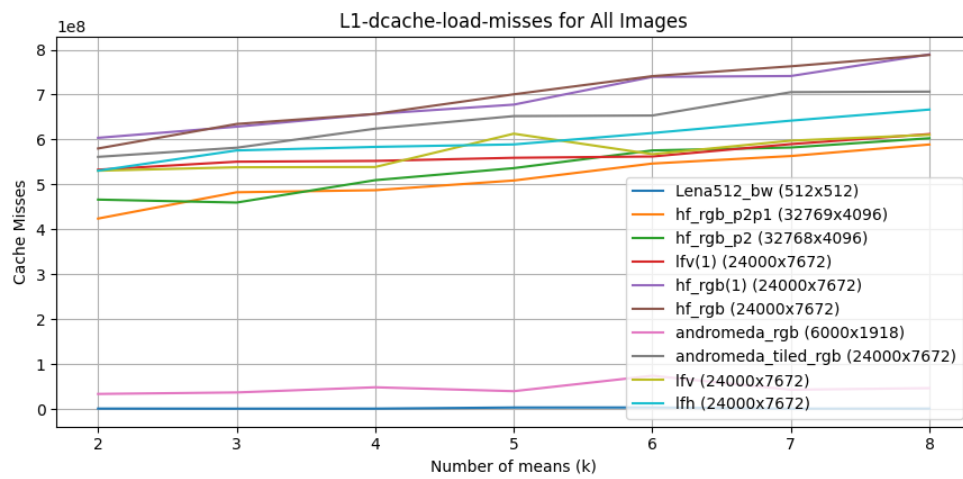


Figure 10: L1 cache misses of the optimized code

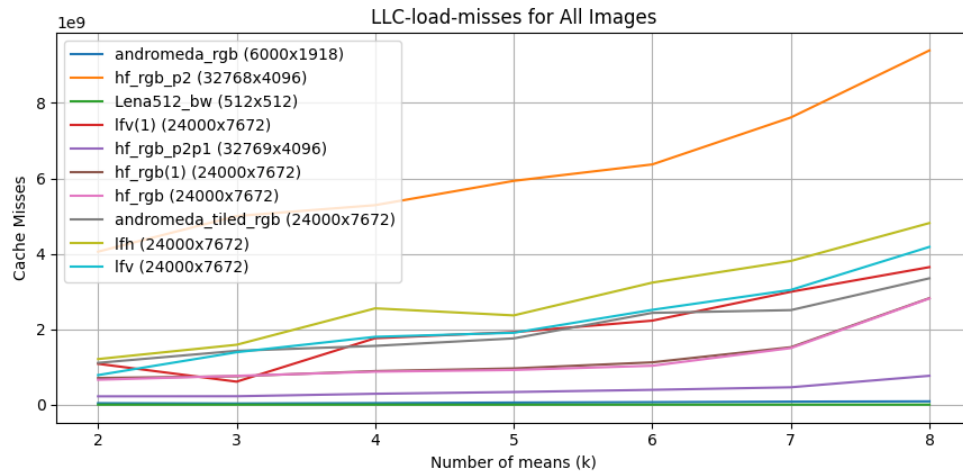


Figure 11: LLC cache misses of the original code

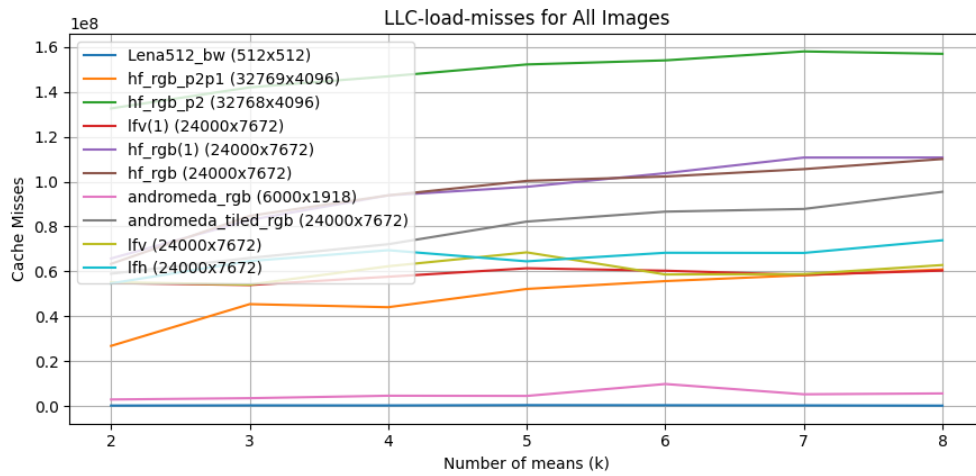


Figure 12: LLC cache misses of the optimized code

Original code executions:

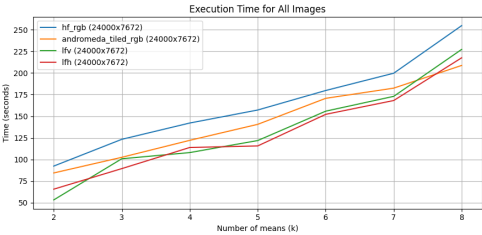


Figure 13: Execution 1

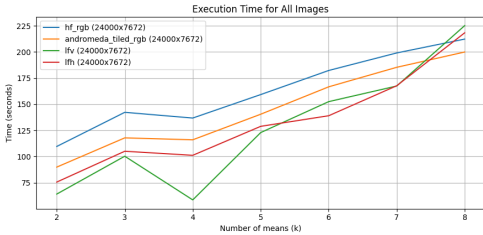


Figure 14: Execution 2

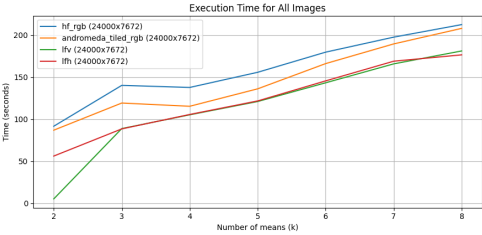


Figure 15: Execution 3

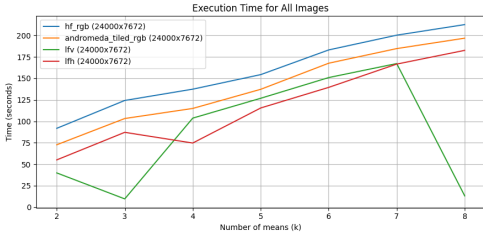


Figure 16: Execution 4

Optimized code executions:

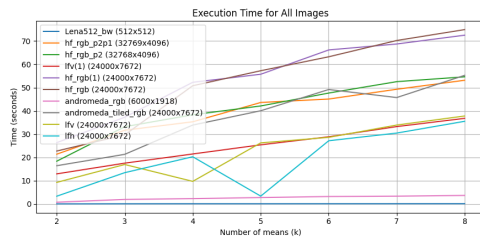


Figure 17: Execution 1

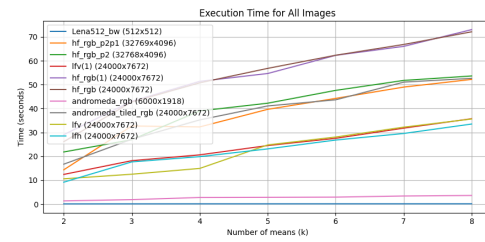


Figure 18: Execution 2

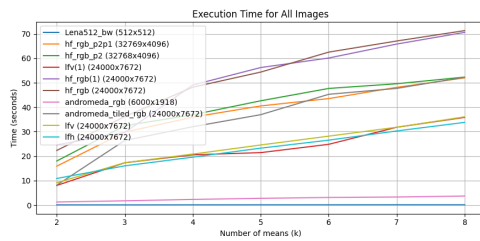


Figure 19: Execution 3

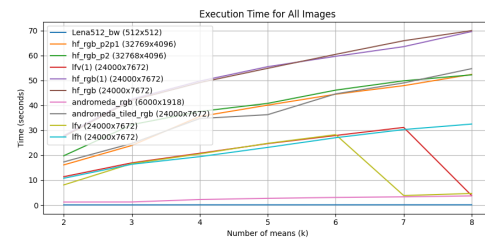


Figure 20: Execution 4

References

- [1] David Bailey. "Unfavorable Strides in Cache Memory Systems (RNR Technical Report RNR92-015)". In: *Scientific Programming - SP* 4 (June 1995), pp. 53–58. DOI: 10.1155/1995/937016.
- [2] Taher Ghazal et al. "Performances of K-Means Clustering Algorithm with Different Distance Metrics". In: (May 2021).
- [3] Jon Inouye et al. "The effects of virtually addressed caches on virtual memory design and performance". In: *SIGOPS Oper. Syst. Rev.* 26.4 (Oct. 1992), pp. 14–29. ISSN: 0163-5980. DOI: 10.1145/142854.142859. URL: <https://doi.org/10.1145/142854.142859>.
- [4] D.A. Patterson and J.L. Hennessy. *Computer Organization and Design MIPS Edition: The Hardware/Software Interface*. The Morgan Kaufmann Series in Computer Architecture and Design. Elsevier Science, 2020. ISBN: 9780128201091. URL: <https://books.google.gr/books?id=9WN9zQEACAAJ>.
- [5] Archana Singh, Avantika Yadav, and Ajay Rana. "K-means with Three different Distance Metrics". In: *International Journal of Computer Applications* 67 (Apr. 2013), pp. 13–17. DOI: 10.5120/11430-6785.
- [6] Pei-Chi Wu and Kuo-Chan Huang. "Case Studies on Cache Performance and Optimization of Programs with Unit Strides". In: *Softw., Pract. Exper.* 27 (Feb. 1997), pp. 167–172. DOI: 10.1002/(SICI)1097-024X(199702)27:2<167::AID-SPE76>3.3.CO;2-H.